

CUSTOMER CHURN PREDICTION SYSTEM

Project Documentation

Prepared by	Nithish G
Date	May 2026
Stack	Python, XGBoost, Streamlit, scikit-learn
Dataset	10,000 customer records, 32 features
Final model	XGBoost - ROC-AUC 80.9%, Recall 75.5%

1. Project Overview

Losing a paying customer is expensive. Winning a new one usually costs several times more than keeping an existing one, so a subscription business gains real value from spotting churn before it happens. This project builds a machine-learning pipeline that reads a customer profile, estimates how likely that customer is to leave, and surfaces the result through a web app that the team can use without writing code. A user enters a customer's details across usage, billing, and support, then receives a churn probability, a risk band (Low, Medium, or High), and a short set of retention actions matched to that customer's weak points.

What the project delivers

- A trained XGBoost classifier that flags customers at risk of churning.
- A tuned decision threshold optimised for recall, since a missed churner costs more than a false alarm.
- A Streamlit interface that lets non-technical staff run predictions in real time.
- Visual outputs: ROC curves, confusion matrices, feature importance, and risk distribution plots.

2. Dataset Description

The source file is `customer_churn_business_dataset.csv`, holding 10,000 customer records across 32 columns. The target is a binary column, `churn`, where 1 marks as customer who left and 0 marks one who stayed.

Class balance

The data is heavily skewed. About 89.8% of customers stayed and only 10.2% churned. This matters because a model can reach 90% accuracy just by predicting “no churn” for everyone, which is useless in practice. SMOTE was later used to address this.

Feature groups

The 32 columns split into four groups:

- **Demographic and account:** `customer_id`, `gender`, `age`, `country`, `city`, `customer_segment`, `tenure_months`, `signup_channel`, `contract_type`
- **Product usage:** `monthly_logins`, `weekly_active_days`, `avg_session_time`, `features_used`, `usage_growth_rate`, `last_login_days_ago`, `email_open_rate`, `marketing_click_rate`

- **Billing and payment:** monthly_fee, total_revenue, payment_method, payment_failures, discount_applied, price_increase_last_3m
- **Support and satisfaction:** support_tickets, avg_resolution_time, complaint_type, csat_score, escalations, nps_score, survey_response, referral_count

Missing values and key patterns

Only complaint_type had missing data, blank for roughly 20.5% of rows. That blank carries meaning: it indicates a customer who never raised a complaint. It was therefore filled with the string “no complaint” rather than dropped or statistically imputed.

Exploration surfaced clear churn patterns. Individual-plan customers churned more than business or enterprise customers. Web signups churned most among signup channels. Month-to-month contract holders churned far more than annual contract holders. Billing complaints carried the highest churn rate of any complaint type, short-tenure customers were over-represented among churners, and low NPS and CSAT scores were strong leading indicators.

3. Steps Followed

The project ran a standard end-to-end workflow, from raw data to a deployable interface.

Step	What was done
1. Data loading and inspection	Loaded the CSV; checked shape, data types, missing counts, duplicates, and class distribution.
2. Exploratory data analysis	Plotted categorical features against churn rate, drew boxplots of numeric features by churn status, and built a correlation heatmap to find multicollinearity.
3. Data cleaning	Filled missing complaint_type values; dropped customer_id and the high-cardinality city column.
4. Feature engineering	Derived six new features from existing columns (detailed in Section 5).
5. Encoding	Label-encoded binary columns; one-hot encoded multi-class columns.
6. Train/test split	Split 80/20 with stratification, preserving the 10:90 churn ratio in both sets.
7. SMOTE	Applied SMOTE to the training set only, balancing the minority class without leaking into the test set.
8. Feature scaling	Fitted a StandardScaler on training data, applied to both sets.
9. Model training	Trained Logistic Regression, Random Forest, and XGBoost; scored each on the test set.
10. Threshold tuning	Used the precision-recall curve to find the F1-optimal threshold per model.
11. Hyperparameter tuning	Ran GridSearchCV with 5-fold cross-validation on each model.
12. Final model selection	Compared tuned models and chose XGBoost on ROC-AUC and recall (see Section 8).
13. Saving model	Saved the model, scaler, encoders, feature list, and threshold as pickle files.
14. Streamlit app	Built the prediction UI with a customer-detail form and instant output.

4. Preprocessing Methods

Each preprocessing decision below was deliberate.

Dropping columns

customer_id is a row identifier with no predictive value, so it was removed. city had too many unique values; one-hot encoding it would have added hundreds of sparse columns that mostly add noise, and country already captures geography at a useful level.

Filling missing values

complaint_type was the only column with gaps, about 2,045 rows. The blank was filled with “no complaint” because a missing complaint is itself a real category that the model should treat differently from a billing or technical complaint.

Encoding

Three two-value columns (gender, discount_applied, price_increase_last_3m) were label-encoded to 0/1 integers. Seven multi-category columns (country, customer_segment, signup_channel, contract_type, payment_method, complaint_type, survey_response) were one-hot encoded, turning each value into its own binary column. Fitted encoders were saved for use at prediction time.

Split and imbalance handling

The data was split 80/20 with random_state 42 and stratification, so both halves kept the 10:90 churn ratio. SMOTE then ran on the training set only, interpolating between real minority examples to bring both classes to 7,183 samples. Running SMOTE before the split would have leaked synthetic data into the test set.

Feature scaling

Tree models are scale-invariant, but Logistic Regression is not, so all three models received scaled features for a fair comparison. The StandardScaler was fitted on training data only, applied to both sets, and saved to disk so the app scales new inputs the same way.

5. Feature Engineering

Six features were derived to give the model clearer signals about behaviour patterns that domain knowledge links to churn risk.

Feature	Formula	Why it helps
revenue_per_month	total_revenue / tenure_months	Normalises revenue by relationship length. The same total revenue over 3 months versus 36 means very different things.
is_inactive	last_login_days_ago > 14	Flags a customer who has not logged in for over two weeks, making inactivity explicit.
has_complaint	complaint_type ≠ 'no complaint'	Collapses all complaint categories into a single yes/no risk signal.
support_intensity	support_tickets / (tenure_months + 1)	Normalises support load by tenure. Five tickets in a month differs from five over three years.
low_engagement	email_open_rate < 0.3	Flags disengaging customers; the 30% threshold came from the observed distribution.
is_high_value	total_revenue > 75th percentile	Marks top-spend customers so the model and recommendation engine can treat them differently.

6. Models Trained

Three classifiers were trained and compared. Starting with a simple baseline and adding complexity shows how much that complexity actually buys.

Logistic Regression (baseline)

Chosen as the starting point for being interpretable, fast, and well understood. It learns a linear boundary separating churners from non-churners. GridSearchCV tuned regularisation strength C (0.01, 0.1, 1, 10), penalty type (L1 vs L2), and solver (liblinear). Best configuration: C 0.1, L1 penalty, liblinear solver.

Random Forest

An ensemble that averages predictions across hundreds of independently trained trees, handling non-linear relationships well. Tuning covered number of trees (100-200), max depth (8-16), minimum samples to split (5-10), and minimum samples per leaf (2-5). Best configuration: 200 trees, depth 16, split 5, leaf 2.

XGBoost (final selection)

Builds trees sequentially, with each tree correcting the errors of the previous ones. It is one of the strongest algorithms for tabular classification and handles imbalanced data well when paired with threshold tuning. Tuning covered estimators (100-200), max depth (3-7), learning rate (0.05, 0.1, 0.2), and subsample ratio (0.8, 1.0). Best configuration: 200 estimators, depth 3, learning rate 0.1, subsample 1.0.

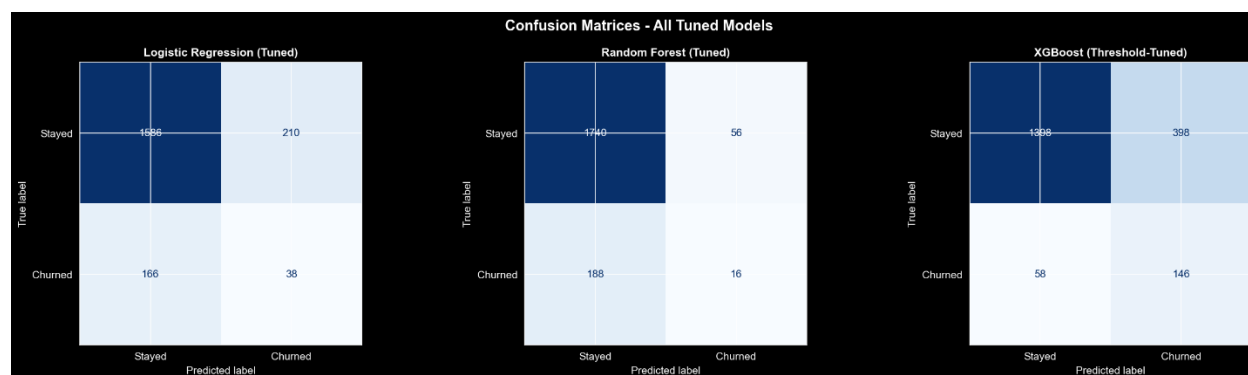
7. Evaluation Results

Each model was scored on the held-out test set of 2,000 records using five metrics, with threshold tuning applied before these numbers were recorded.

- **Accuracy:** share of all predictions that were correct.
- **Precision:** of customers flagged as likely to churn, how many actually did.
- **Recall:** of customers who actually churned, how many were caught.
- **F1-score:** harmonic mean of precision and recall.
- **ROC-AUC:** how well the model separates the classes across all thresholds; 0.5 is random, 1.0 is perfect.

Model	Acc.	Prec.	Recall	F1	ROC-AUC	Thresh.
Logistic Regression	81.2%	15.3%	18.6%	16.8%	61.7%	0.30
Random Forest	87.8%	22.2%	7.8%	11.6%	76.3%	0.334
XGBoost (selected)	75.2%	25.7%	75.5%	38.4%	80.9%	0.135

Thresholds are the F1-optimal cut-offs from each precision-recall curve.

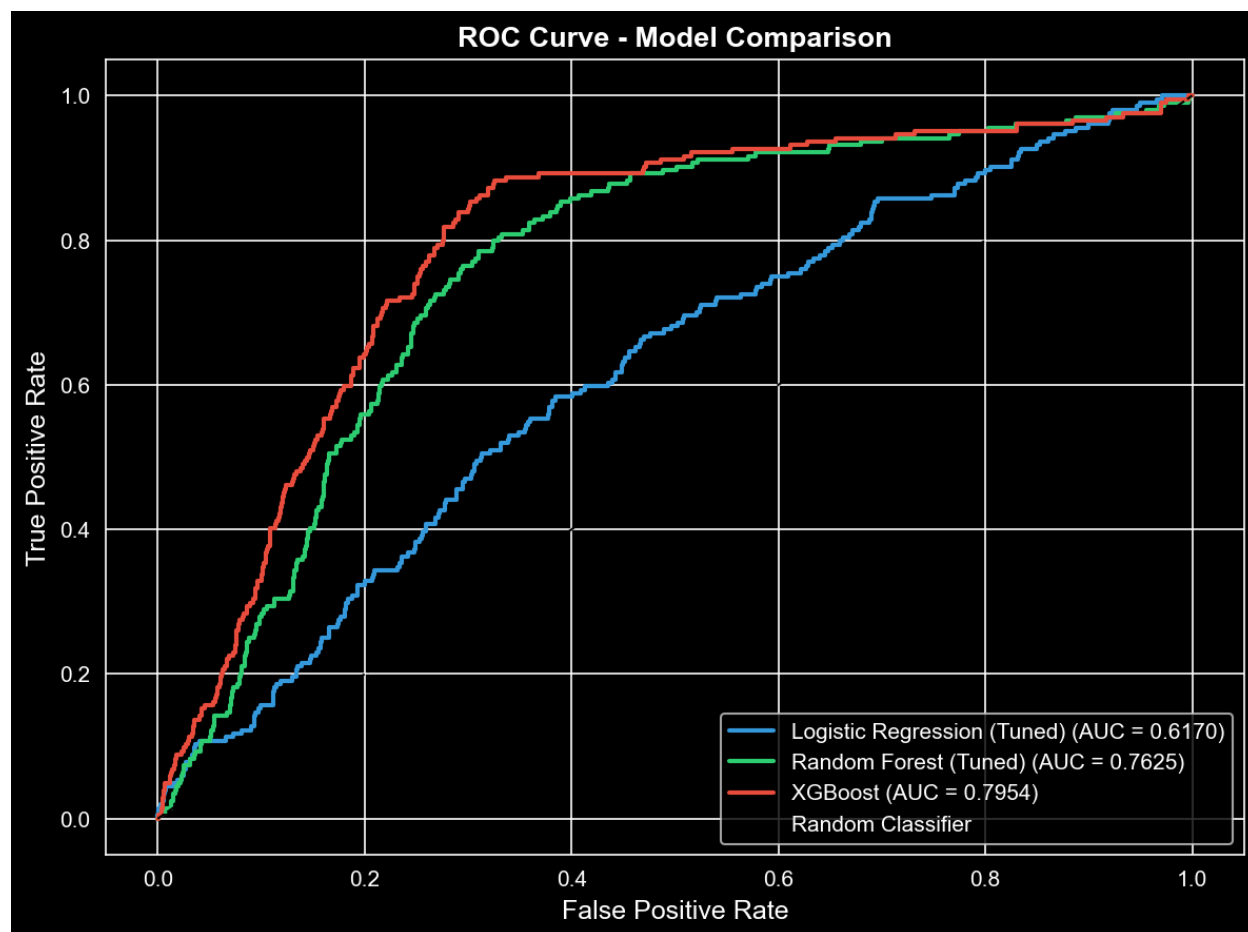


Reading the numbers

Logistic Regression was weak throughout. Its ROC-AUC of 61.7% is barely above a coin toss, and recall of 18.6% means it missed more than four in five real churners.

Random Forest looked strong on accuracy at 87.8%, but recall fell to 7.8% after tuning. It was effectively predicting “no churn” for nearly everyone and being rewarded by accuracy, which is a failure mode for churn detection.

XGBoost traded some accuracy, at 75.2%, but caught 75.5% of real churners and posted the highest ROC-AUC at 80.9%, confirming it had genuinely learned to separate the classes.



8. Final Model Selection

XGBoost was chosen as the production model. The decision rests on one business question: which mistake costs more, wrongly flagging a loyal customer, or missing a real churner?

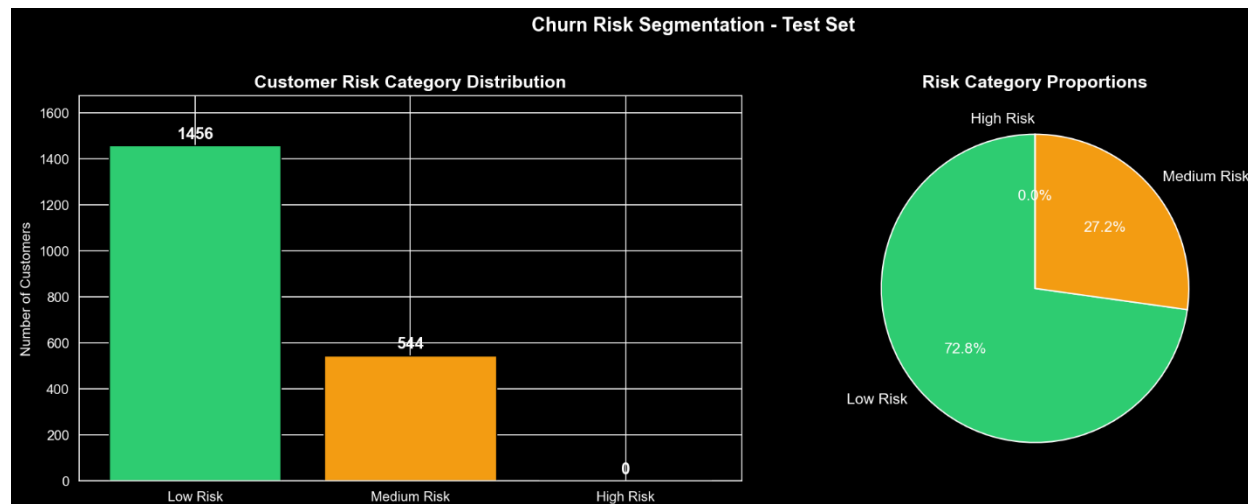
For most subscription businesses the missed churner is worse. A loyal customer flagged by mistake gets, at worst, an unneeded discount or a check-in call. A missed churner leaves quietly, and recovering them costs far more than the intervention would have. That makes recall the primary metric here, with ROC-AUC as the secondary measure. XGBoost led on both.

Why not Random Forest

It had higher accuracy and precision, but recall of 7.8% made it functionally useless for churn detection. A model that misses 92% of churners offers a false sense of security rather than a retention tool.

Why not Logistic Regression

Its ROC-AUC of 61.7% showed it had not captured the non-linear structure in the data, and its recall was poor. It was kept in the artefacts folder only for comparison.



The default 0.5 boundary did not work for any model on this imbalanced data. For XGBoost the F1-optimal threshold was 0.135. At that cut-off the model catches 75.5% of churners at 25.7% precision, meaning roughly one in four flagged customers truly churns, a useful signal for targeted outreach.

9. Risk Category Logic

The model outputs a raw churn probability between 0 and 1. That number is mapped to one of three risk bands so the team can act on it.

Risk band	Probability	Meaning
Low Risk	< 0.135	Little or no churn signal. Monitor normally.
Medium Risk	0.135 - 0.60	Elevated risk. Some warning signs worth addressing.
High Risk	≥ 0.60	Strong churn signal. Immediate intervention recommended.

The lower boundary of 0.135 is the F1-optimal threshold, the point where the model begins predicting churn, so any customer past it is at least Medium Risk. The upper boundary of 0.60 separates strong signals from borderline cases, since high-risk customers need a more urgent response.

On the test set no customer exceeded 0.60, so every flagged customer fell into the Medium band. This fits a model that flags risk early rather than waiting for certainty.

10. Recommendation Logic

Knowing a customer might churn is only half the task. The app turns each prediction into a retention plan, built from the risk band and the specific signals in the customer's input.

High Risk - act within 24 to 48 hours

- Assign a dedicated Customer Success Manager for hands-on support.
- Proactively offer a loyalty discount or pricing adjustment.
- Fast-track any open support tickets to immediate resolution.
- Schedule a call or video meeting within 48 hours.

Medium Risk - act within 1 to 2 weeks

- Send a targeted satisfaction survey to find specific pain points.
- Offer a product walkthrough or onboarding refresh.
- Consider a modest discount or feature upgrade as goodwill.
- Schedule a quarterly business review or check-in call.

Low Risk - maintain and nurture

- Include in regular email nurture campaigns with relevant content.
- Enrol in the referral programme to deepen engagement.
- Recognise tenure milestones with anniversary emails or loyalty badges.
- Identify upsell or cross-sell opportunities suited to usage level.

Signal-based personalisation

Beyond the risk band, the app reads specific input values to add targeted advice. Unresolved support tickets trigger a fast-track note, a very low email open rate triggers a re-engagement note, and a below-average NPS or CSAT score triggers a survey suggestion. The output reads as a tailored action plan rather than a generic checklist.

11. UI Usage Instructions

The prediction interface is a Streamlit web app. It runs locally and needs no cloud account or API key.

Starting the application

Open a terminal in the project root and run:

```
venv\Scripts\streamlit.exe run app/app.py
```

On first run, Streamlit opens a browser tab at <http://localhost:8501>. Later runs reuse the same port.

Filling in the customer form

The form has three columns, each covering one part of the customer profile.

- **Column 1, Demographics and account:** age, gender, country, customer segment, tenure, contract type, and signup channel.
- **Column 2, Usage and billing:** login frequency, active days, session time, features used, growth rate, days since last login, monthly fee, total revenue, payment method, and payment failures.
- **Column 3, Support and engagement:** discount status, price-increase flag, support tickets, resolution time, complaint type, CSAT, escalations, email open rate, marketing click rate, NPS, survey response, and referral count.

Running a prediction

With all fields filled, click Predict Churn. The app runs the input through the same pipeline used in training (feature engineering, encoding, scaling) and passes it to the XGBoost model.

Reading the output

- Churn Probability: a percentage from 0 to 100; higher means more likely to churn.
- Risk Category: Low, Medium, or High, shown with a colour-coded badge.
- Model Verdict: a plain statement, either Likely to Churn or Likely to Stay.
- Progress Bar: a visual reading of the churn probability.
- Retention Recommendations: an action plan matched to this customer's profile.

Retraining the model

To retrain on updated data, run `python streamlit_code.py`. This regenerates every pickle file in `models/`. Because the app uses `@st.cache_resource`, the new model loads on the next app restart.

12. Challenges Faced and How They Were Handled

Severe class imbalance (90:10)

With nine non-churners per churner, most models learn to predict “no churn” and still score 90% accuracy. SMOTE oversampled the minority class to a 50:50 balance, applied only after the train/test split to avoid leaking synthetic samples into the test set. Threshold tuning then shifted the operating point further toward recall.

Threshold versus accuracy trade-off

At the default 0.5 threshold, models looked acceptable on accuracy but were near-useless on recall, with XGBoost effectively at zero. Plotting each precision-recall curve found the F1-maximising threshold; for XGBoost this was 0.135, which lifted recall to 75.5% while holding precision at a workable 25.7%.

High-cardinality categorical columns

city held hundreds of unique values; one-hot encoding it would have produced hundreds of sparse columns. It was dropped, with country retaining geography at a useful level. A target-encoding approach could revisit this with more data.

Working-directory path issues in the app

The original `app.py` used relative paths like `models/xgboost_model.pkl`, which broke unless Streamlit launched from the project root. The path logic was rewritten with `pathlib` to resolve paths from the app file itself (`ROOT = Path(__file__).resolve().parent.parent`), so the app works from any directory.

Missing complaint_type data

Roughly 20.5% of rows had no complaint_type. Dropping them would have lost one customer in five. The blank was filled with “no complaint”, which keeps every row and adds a genuine category the model can learn from.

Logistic Regression and non-linear patterns

After tuning, Logistic Regression's ROC-AUC plateaued at 61.7%, and pushing the threshold lower to gain recall destroyed precision. Linear models were accepted as a poor fit, and the project moved to ensemble methods that capture the feature interactions Logistic Regression cannot represent.
